Documento Técnico de Especificación y Arquitectura: Sincronizador de Escalado (CloudRunClientWrapper)

Fecha de actualización: 16 de Octubre de 2026 | Autor / Sistema: Motor de Escalado CREMA

1. Propósito y Responsabilidades del Módulo

El componente **CloudRunClientWrapper** actúa como el conector y ejecutor de órdenes de escalado para servicios en Google Cloud Run v2. Sus responsabilidades principales son:

- **Inspección en Tiempo Real:** Consultar directamente en Google Cloud Run v2 el valor real del parámetro --min (Service Scaling / manual_instance_count).
- **Auditoría Transparente:** Registrar en la bitácora de logs el estado real del servicio en cada ciclo de revisión.
- **Evaluación de Discrepancia:** Comparar el valor del parámetro --min configurado en GCP frente a las instancias recomendadas calculadas por el motor.
- **Despacho de Órdenes a Pub/Sub:** Publicar un mensaje JSON con los parámetros min y min_instances al tópico de Pub/Sub cuando exista cualquier diferencia entre el estado real y el deseado.

2. Diferencia Crítica de Parámetros: --min vs --min-instances

Google Cloud Run gestiona dos niveles de escalado mínimos independientes:

Parámetro CLI / UI	Nivel de Ámbito	Mapeo en API Java v2	Descripción
--min	Service Scaling (Nivel Servicio)	service.getScaling().getManualInstanceCount()	Aplica a nivel global de servicio. Es el parámetro principal evaluado y corregido por este módulo.
--min-instances	Revision Scaling (Por Revisión)	service.getTemplate().getScaling().getMinInstanceCount()	Aplica de forma específica a la plantilla de una revisión individual. Utilizado como fallback en la inspección.
Nota de API	Sincronización dual	lastSentInstancesCache	Mantiene la coherencia en memoria entre llamadas seguidas para prevenir envíos duplicados a Pub/Sub.

3. Lógica de Evaluación y Matriz de Decisiones

3.1. Fórmula de Comparación

En cada ciclo de ejecución, el método updateServiceMinInstances evalúa la siguiente condición lógica:

```
¿ Es  "Valor Real de --min en GCP" == "Instancias Recomendadas Calculadas" ?
 ├── SI ──► Omitir envío a Pub/Sub. Registra: "[SCALER] Sin cambios requeridos..."
 └── NO ──► Forzar actualización. Registra: "[SCALER] Discrepancia detectada..." y
publica a Pub/Sub.
```

3.2. Matriz de Casos de Uso

Escenario de Negocio	Estado Real --min	Instancias Recomendadas	Resultado Comparación	Acción Ejecutada
Tráfico Estable	1	1	Iguales (1 == 1)	No se envía mensaje. Se mantiene la caché.
Caída de Tráfico (Desescalado a cero)	1	0	Diferentes (1 != 0)	Se envía orden min=0 a Pub/Sub.
Modificación Manual / Ajuste Forzado	2 (Cambio manual)	0 (Sin tráfico)	Diferentes (2 != 0)	Se corrige el punto ciego: Se publica inmediatamente min=0 a Pub/Sub.
Incremento de Demanda	0	5	Diferentes (0 != 5)	Se envía orden min=5 a Pub/Sub.

4. Integración con el Cálculo de Métricas y Flujo E2E

Aunque CloudRunClientWrapper no procesa las métricas de CPU directamente, colabora en el flujo completo del sistema CREMA:

Paso 1. Recopilación de Métricas (MetricsService.java): Consulta en Cloud Monitoring la CPU consumida y las instancias activas (state="active").

Paso 2. Cálculo Matemático del Motor (CpuScaler): Instancias Requeridas = Math.ceil((CPU_Actual / CPU_Target) * Instancias_Activas)

Paso 3. Inspección y Sincronización (CloudRunClientWrapper.java): Ejecuta getServiceMinParam(...) y emite en el LOG: status actual de instancias del servicio (parametro --min) = X. Compara X contra las Instancias Requeridas. Si difieren, despacha el JSON a Pub/Sub.

Paso 4. Ejecución en GCP (Cloud Function): Consume el mensaje de Pub/Sub y aplica el cambio final en Cloud Run.

5. Formato del Mensaje Emitido a Pub/Sub

El payload JSON enviado al tópico configurado en la variable de entorno PUB_SUB_TOPIC mantiene compatibilidad total agregando ambos identificadores:

```
{
  "service": "nombre-del-servicio",
  "project": "id-del-proyecto",
  "region": "us-central1",
  "min": 0,
  "min_instances": 0
}
```